

MediSphere

Hospital Management System

Object-Oriented Programming · Python 3 + Tkinter

Subject Code : MC122 — Object-Oriented Programming 2025-26

Student 1 : Havish Trivedi — ID : 202503013

Student 2 : Sneha C Patel — ID : 202503007

Language : Python 3

Framework : Tkinter + ttk (Desktop GUI — cross-platform)

OOP Design

Tkinter GUI

Bill & File Save

Emergency Module

QR Payment

1 Project Overview

MediSphere is a fully functional desktop **Hospital Management System** built with Python 3 and Tkinter. It simulates a real-world hospital workflow — patient registration, doctor scheduling, appointment booking, itemised billing with automatic file download, and 24×7 emergency dispatch. The codebase is architected entirely around the five core **OOP principles**: Inheritance, Encapsulation, Polymorphism, Abstraction, and Composition.

Language	Python 3
GUI Framework	Tkinter + ttk (cross-platform desktop)
Subject	MC122 — Object-Oriented Programming (2025-26)
Students	Havish Trivedi 202503013 Sneha C Patel 202503007
Core Classes	Person · Patient · Doctor · Appointment · Bill · Hospital · HospitalApp

2 OOP Architecture & Class Hierarchy

Person is the base class. **Patient** and **Doctor** inherit it via `super().__init__()`. Supporting classes (Appointment, Bill, Hospital, HospitalApp) use *composition* — they hold references to Patient/Doctor objects rather than inheriting.

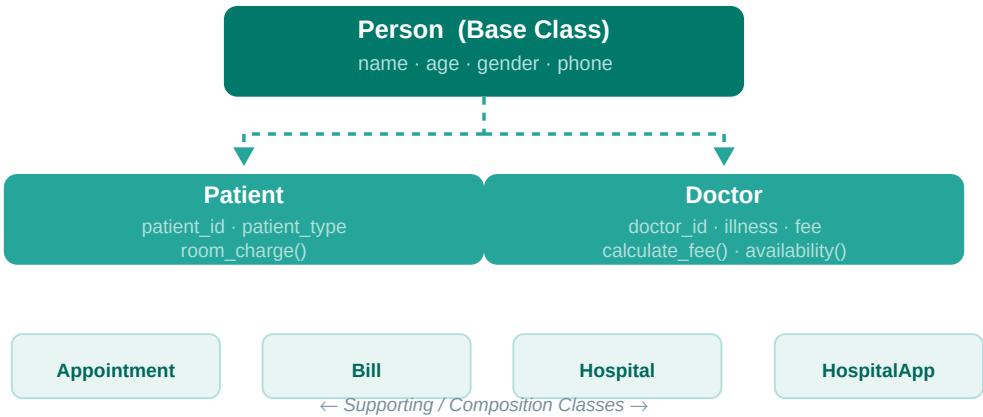


Figure 1 — Class Hierarchy & Composition Diagram

Class	Relationship	Key Attribute / Method
Appointment	Composition — holds Patient + Doctor	appointment_id · slot · status
Bill	Composition — holds Patient + Doctor	total_amount() = appt+doctor+room+service
Hospital	Aggregation — central data store	patients{} · doctors{} · appointments[] · bills[]
HospitalApp	Controller — owns Hospital instance	4 tab-pages · all event handlers

3 Class-by-Class Code Explanation

3.1 Person — Base Class (Abstraction & DRY Principle)

Root of the inheritance tree. Stores **name**, **age**, **gender**, **phone** — defined once, reused by all sub-classes. `int(age)` enforces numeric type at construction.

```
class Person:
    def __init__(self, name, age, gender, phone):
        self.name = name; self.age = int(age) # enforces numeric
        self.gender = gender; self.phone = phone
```

3.2 Patient — extends Person (Inheritance + Encapsulation)

Calls `super().__init__()` for constructor chaining, then adds **patient_id** (PAT001... via class counter) and **patient_type**. **room_charge()** returns 1000 for Inpatients, 0 for Outpatients — polymorphic behaviour.

```
class Patient(Person):
    count = 1 # class variable – shared counter
    def __init__(self, name, age, gender, phone, patient_type):
        super().__init__(name, age, gender, phone) # constructor chaining
        self.patient_id = 'PAT' + str(Patient.count).zfill(3)
        Patient.count += 1; self.patient_type = patient_type
    def room_charge(self): # polymorphic method
        return 1000 if self.patient_type == 'Inpatient' else 0
```

3.3 Doctor — extends Person (Polymorphism)

Adds **illness**, **specialization**, **fee**, **doctor_type**, **slots**. **calculate_fee()** applies Rs.+300 for Specialists. **availability()** formats slot list to a readable string.

```
def calculate_fee(self): # Polymorphic – varies by doctor_type
    return self.fee + 300 if self.doctor_type == 'Specialist' else self.fee
def availability(self):
    return ', '.join(self.slots) # ['9 AM','10 AM'] -> '9 AM, 10 AM'
```

3.4 Appointment & Bill — Composition Pattern

Both classes receive fully-constructed Patient and Doctor objects — composition, not inheritance. Bill's **total_amount()** aggregates Appointment Fee (Rs.100) + Doctor Fee + Room Charge + Service Charge.

```
class Bill:
    def __init__(self, patient, doctor, illness, slot, service_charge):
        self.patient = patient; self.doctor = doctor # composition
        self.appointment_charge = APPOINTMENT_CHARGE # 100
        self.doctor_fee = doctor.calculate_fee() # uses polymorphism
        self.room_charge = patient.room_charge() # uses polymorphism
        self.service_charge = float(service_charge)
    def total_amount(self):
        return self.appointment_charge + self.doctor_fee + self.room_charge + self.service_charge
```

3.5 Hospital — Central Data Store (Encapsulation)

Holds **patients** and **doctors** in dictionaries (O(1) lookup by ID) and lists for appointments / bills / emergencies. All external writes go through `add_patient()` and `add_doctor()` — clean encapsulation.

4 GUI Design — HospitalApp (Tkinter)

HospitalApp is the MVC controller. It builds a teal-themed Notebook with four tabs, each delegated to its own builder method:

Tab	Purpose	Key Widgets
Patients	Register new patients	Entry fields · Gender & Type Combobox · Live Treeview
Doctors	Browse available doctors	Read-only Treeview (ID, name, specialization, fee, slots)
Appointments & Bills	Book slots & generate invoice	Cascading Comboboxes · Text output panel · QR Canvas
Emergency	Rapid emergency dispatch	Patient & doctor combos · Ambulance Yes/No toggle

Dynamic Cascading Dropdowns

Choosing an illness fires `update_doctor_combo()` — showing only matching doctors. Choosing a doctor fires `update_slot_combo()` — showing only that doctor's slots. Prevents all impossible combinations.

```
self.illness_combo.bind('<<ComboboxSelected>>', self.update_doctor_combo)
self.doctor_combo.bind(' <<ComboboxSelected>>', self.update_slot_combo)

def update_doctor_combo(self, event=None):
    illness = self.illness_combo.get()
    self.doctor_combo['values'] = [
        d.doctor_id+ ' - Dr.'+d.name for d in self.hospital.doctors.values()
        if d.illness == illness]          # filter by selected illness
```

5 Key Features & Functionalities

Patient Registration	Name, age, phone, gender, type (Outpatient/Inpatient). Auto-assigns PAT001... ID. Instantly added to live Treeview and all combo boxes.
Doctor Directory (16 doctors, 10 specialisations)	Pre-loaded across Cardiology, ENT, Dermatology, Neurology, Gynecology, Dental, Pediatrics, Orthopedic, Physiotherapy, General Medicine. Fee already includes +Rs.300 Specialist surcharge.
Smart Appointment Booking	Patient → illness → filtered doctor → available slot. Creates Appointment object, prints formatted confirmation with ID, doctor fee, and payment status. Shows popup dialog.
Itemised Bill + Direct File Download	Full invoice: Appointment Fee (Rs.100) + Doctor Fee + Room Charge (Rs.1000 Inpatients) + custom Service Charge. Previewed on-screen, then Save As dialog auto-opens — bill saved as .txt to user's computer.
QR Code Payment Panel	15×15 binary matrix drawn pixel-by-pixel on Tkinter Canvas — no image library needed. Represents Rs.100 appointment fee payment.
Emergency Dispatch (24×7)	Patient + illness + doctor + ambulance option. Calculates Emergency Fee (Rs.2000) + Doctor Fee. Outputs Urgent status confirmation with full cost breakdown.
Pre-loaded Sample Data	<code>load_sample_data()</code> registers 4 patients + 16 doctors on startup — fully demo-ready, zero manual entry.

6 OOP Concepts Demonstrated

OOP Concept	Where Used in MediSphere	Code Example
Inheritance	Patient and Doctor extend Person	super().__init__(name, age, gender, phone)
Encapsulation	ID counters & fee logic inside classes	Patient.count += 1 / Bill.total_amount()
Polymorphism	room_charge() & calculate_fee() by type	Inpatient→1000 Specialist→fee+300
Abstraction	Hospital hides data layer from GUI	add_patient() / add_doctor()
Composition	Appointment/Bill hold Patient/Doctor refs	self.patient = patient (object reference)
Class Variables	Shared auto-increment ID counters	Patient.count / Doctor.count / Bill.count
Constructor Chaining	super().__init__() in derived classes	Avoids duplicate attribute assignments
Event-Driven GUI	Combo bindings → cascading filter logic	bind('<>', handler)

7 Sample Bill Output

Inpatient Amit Kumar · Specialist Cardiologist Dr. Arjun Rao · Slot: 3 PM

```
===== MEDISPHERE
Bill ID : BIL001      Date : 30-04-2025      Patient : Amit Kumar (PAT003)
Type   : Inpatient    Phone: 9988776655      Illness : Heart Problem
Doctor : Dr. Arjun Rao Specialization : Cardiology Slot : 3 PM
-----
Appointment Fee : Rs. 100.00
Doctor Fee      : Rs. 1300.00 (base 1000 + 300 Specialist surcharge)
Room Charge     : Rs. 1000.00 (Inpatient bed)
Service Charge  : Rs. 500.00 (additional services)
-----
TOTAL AMOUNT    : Rs. 2900.00      Payment Status : Paid
=====
Thank you for visiting MediSphere.
```

8 Summary & Conclusion

MediSphere successfully demonstrates all five core OOP principles in a realistic working application. The **inheritance chain** (Person → Patient/Doctor) eliminates duplication; **encapsulation** keeps counters and fee logic inside their classes; **polymorphism** lets room_charge() and calculate_fee() behave differently per type; **abstraction** shields the GUI from the Hospital data layer; and **composition** wires Appointment and Bill to live objects.

Beyond academics, the project delivers a production-quality Tkinter GUI with event-driven cascading dropdowns, formatted bill generation with native Save-As file download, a canvas-drawn QR code, and a 24×7 emergency dispatch module — making it both a rigorous OOP showcase and a genuinely usable application.